ФЕДЕРАЛЬНОЕ АГЕНТСТВО МОРСКОГО И РЕЧНОГО ТРАНСПОРТА

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«ГОСУДАРСТВЕННЫЙ МОРСКОЙ УНИВЕРСИТЕТ ИМЕНИ АДМИРАЛА Ф.Ф. УШАКОВА»

ТРАНСПОРТНЫЙ КОЛЛЕДЖ

ОТЧЁТ

по лабораторной работе № 1
по дисциплине «Разработка кода информационных систем»

Тема: «Система контроля версий Git»

Выполнил:
курсант группы 3831
Киснер Артем Андреевич

Проверил:
преподаватель _____
«____» _____ 2026 г.
Оценка: _____

Новороссийск – 2026

Лабораторная работа № 1

Система контроля версий Git

Цель работы: изучить основы системы контроля версий Git: создание репозитория, фиксацию изменений, игнорирование файлов, работу с ветками, слияние и разрешение конфликтов, работу с удалённым репозиторием на GitHub.

Программное обеспечение: Git 2.55 for Windows, Visual Studio Code (встроенный терминал PowerShell), браузер Google Chrome, сервис GitHub.

Репозиторий: https://github.com/Masster9/git-practice

Задание

1) Создать новый локальный репозиторий.

2) Создать два текстовых файла, добавить их в репозиторий и сделать первый коммит.

3) Добавить файл .gitignore с шаблоном для игнорирования определённых файлов.

4) Изменить один из отслеживаемых файлов и сделать второй коммит.

5) Создать новую ветку, переключиться в неё, изменить другой файл и сделать коммит.

6) Вернуться в основную ветку и выполнить слияние с новой веткой (fast-forward, без конфликтов).

7) Сделать ещё одно изменение в основной ветке и зафиксировать его коммитом.

8) Перейти в новую ветку, изменить тот же файл, что и в основной ветке, выполнить коммит и слить ветки. Разрешить возникший конфликт вручную и сделать коммит.

9) Просмотреть историю коммитов с отображением ветвлений.

10) Создать удалённый репозиторий на GitHub, связать его с локальным и отправить в него все коммиты.

11) На GitHub создать файл README.md с кратким описанием проекта.

12) Получить изменения с GitHub командой pull, чтобы скачать файл README.md.

Ход работы

Шаг 1. Создание локального репозитория

Командами git config --global user.name и user.email задано имя и адрес автора коммитов. В пустой папке git-practice, открытой в VS Code, выполнена команда git init — создан пустой репозиторий (служебная папка .git). Команда git status показывает ветку master без коммитов (рисунок 1).

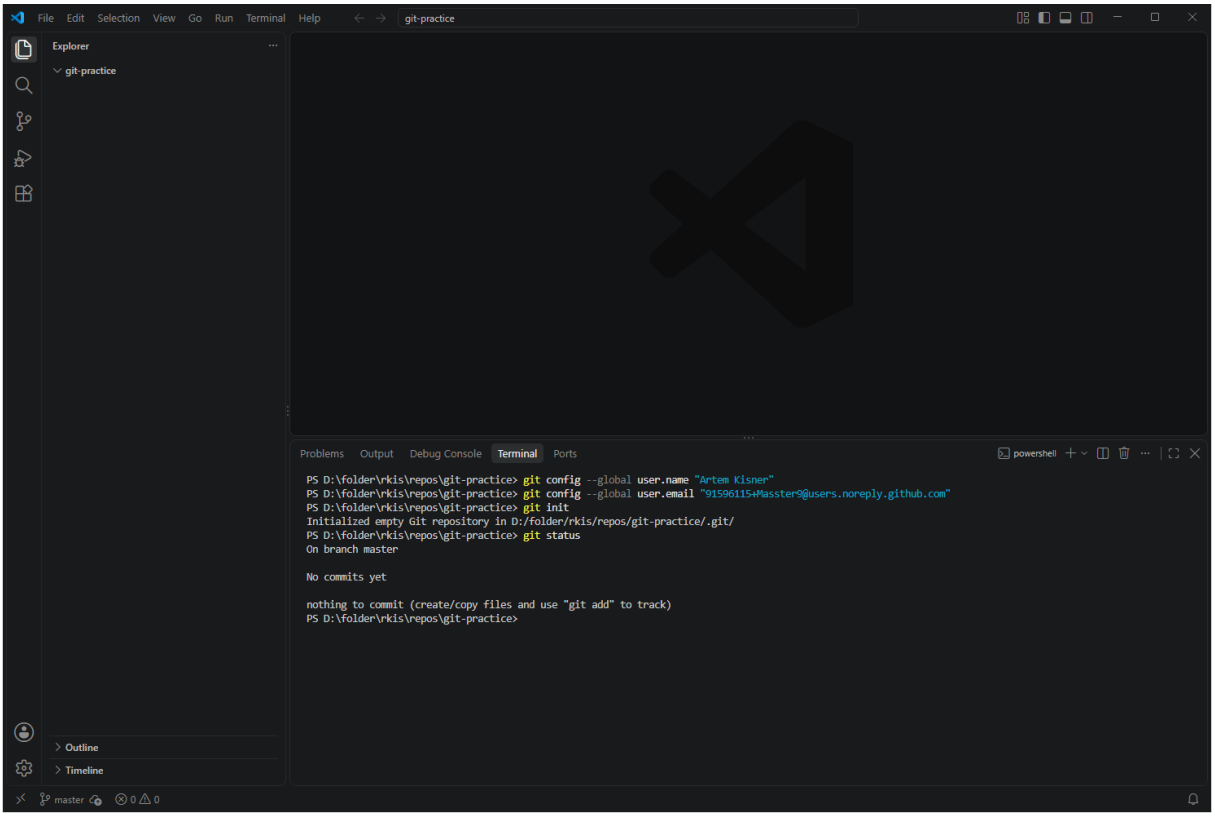


Рисунок 1 – Настройка Git и создание репозитория командой git init

Шаг 2. Первый коммит

Созданы файлы notes.txt (заметки по Git) и plan.txt (план работы). Командой git add они добавлены в индекс, командой git commit -m зафиксированы первым коммитом. После коммита git status сообщает, что рабочая копия чистая (рисунок 2).

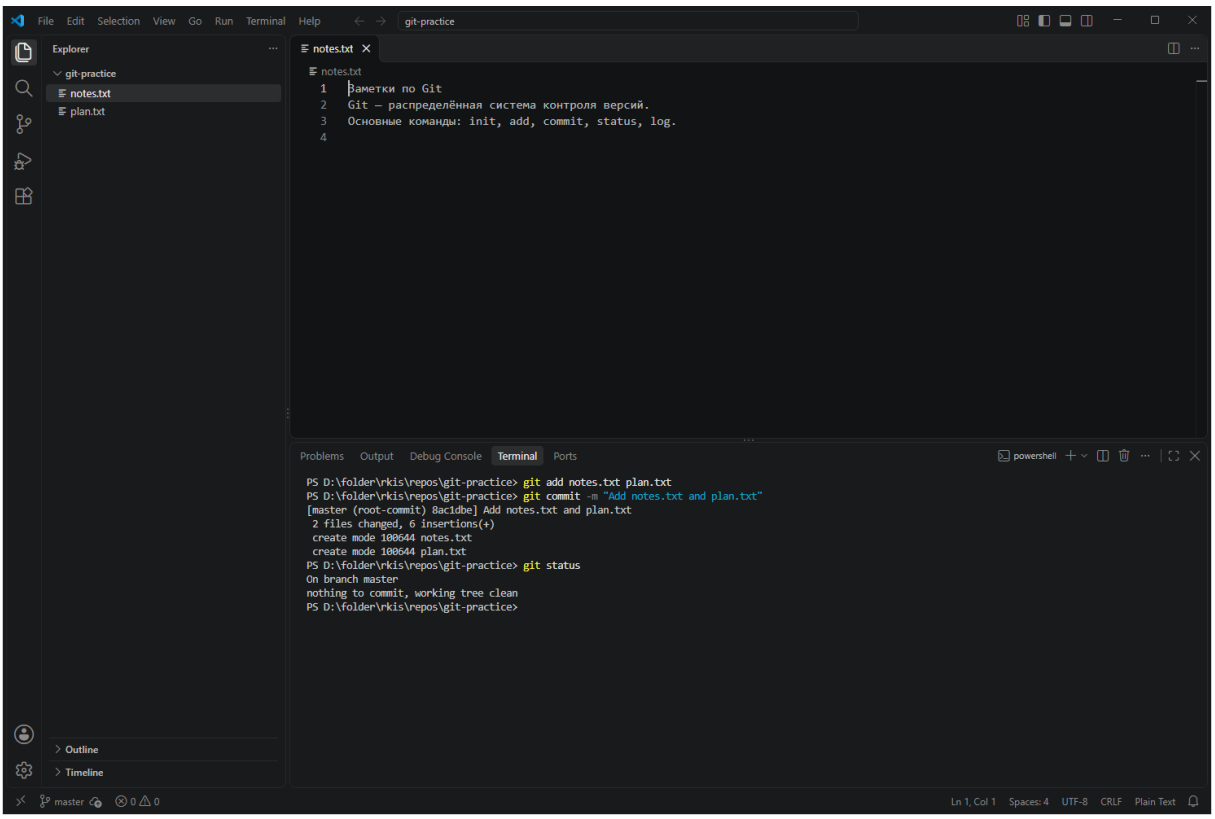


Рисунок 2 – Добавление файлов в индекс и первый коммит

Шаг 3. Файл .gitignore

Создан файл журнала debug.log и файл .gitignore с шаблонами *.log (все файлы журналов) и temp/ (временная папка). Команда git status показывает как неотслеживаемый только .gitignore — файл debug.log Git уже не видит. После коммита .gitignore команда git status --ignored выводит debug.log в списке игнорируемых файлов; в проводнике VS Code он отображается серым (рисунок 3).

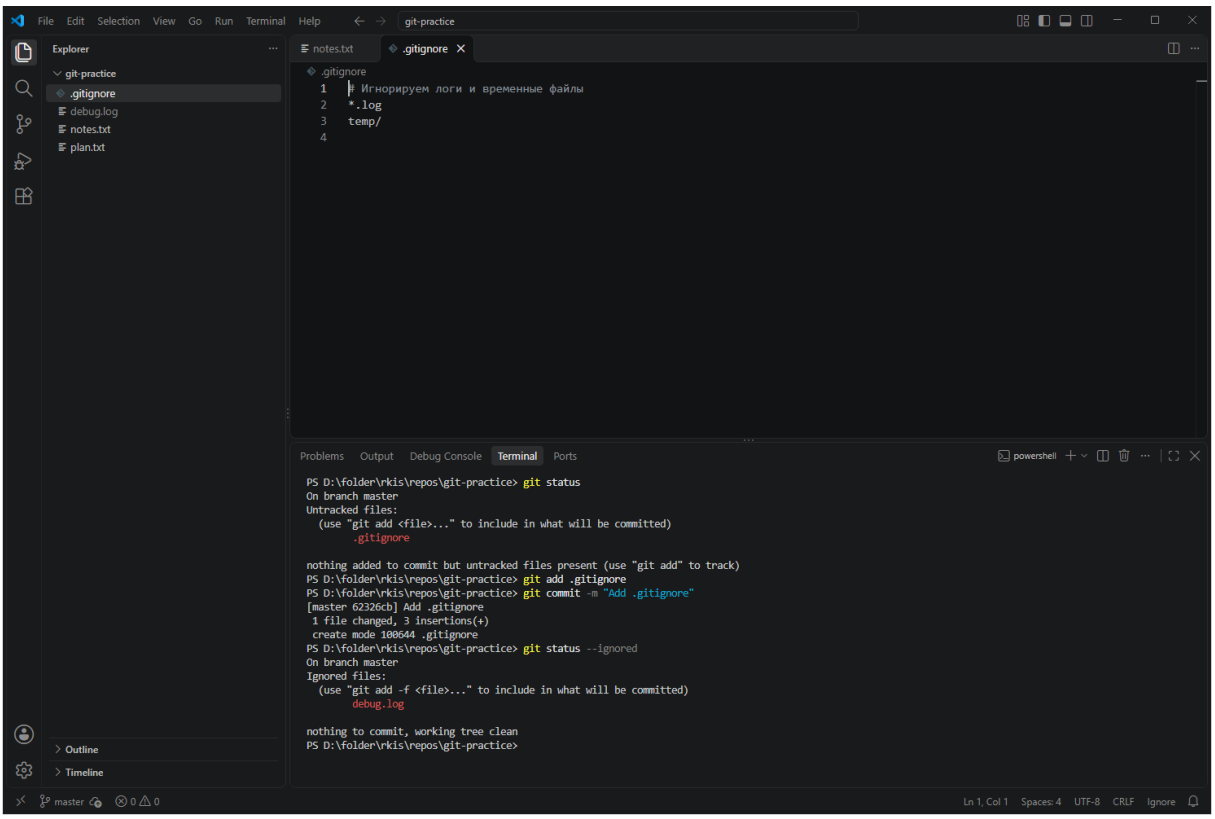


Рисунок 3 – Файл .gitignore: debug.log игнорируется

Шаг 4. Изменение файла и второй коммит

В notes.txt добавлена строка. Команда git diff показывает изменение (добавленная строка отмечена знаком «+»). Изменение зафиксировано командой git commit -am, которая сама добавляет в индекс изменённые отслеживаемые файлы (рисунок 4).

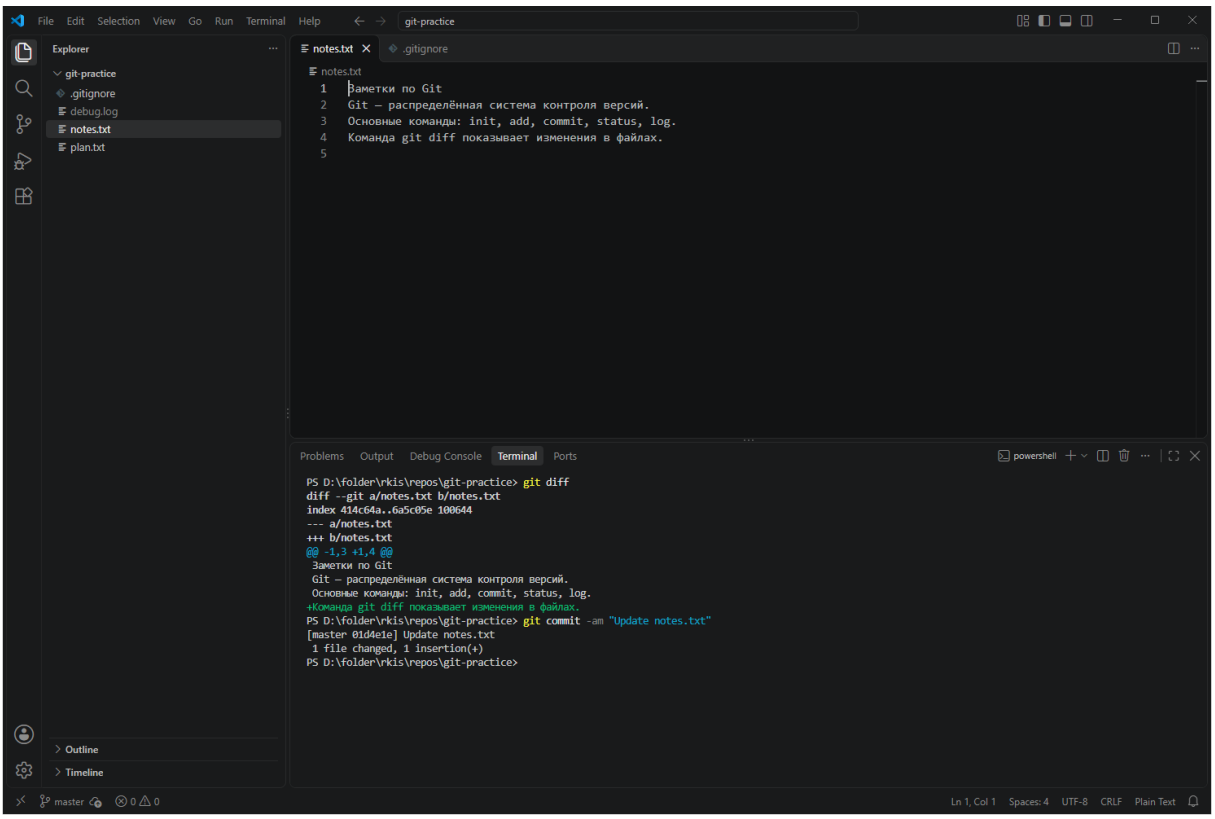


Рисунок 4 – Просмотр изменений git diff и второй коммит

Шаг 5. Новая ветка

Команда git checkout -b feature создала ветку feature и сразу переключила на неё. В ветке изменён файл plan.txt (добавлены пункты 3 и 4) и сделан коммит. Команда git branch показывает две ветки, текущая отмечена звёздочкой (рисунок 5).

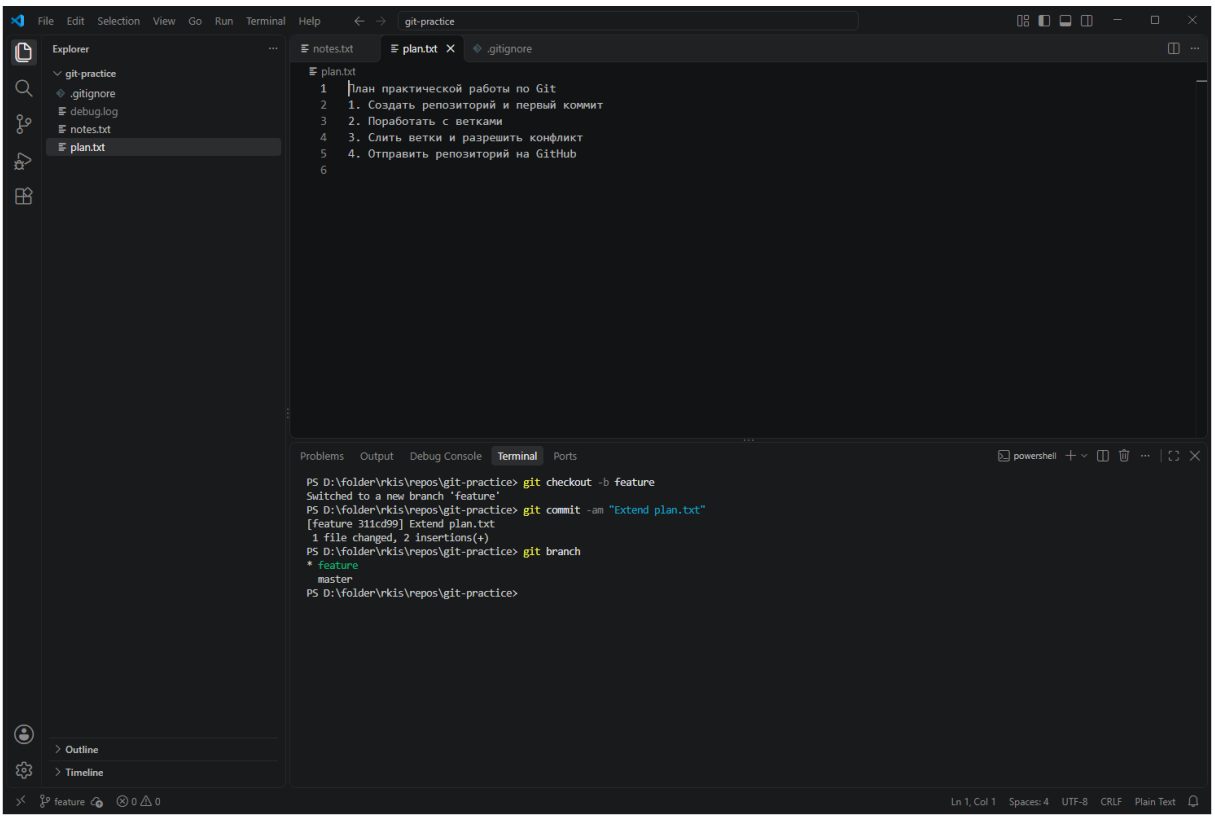


Рисунок 5 – Создание ветки feature и коммит в ней

Шаг 6. Слияние fast-forward

После git checkout master выполнено git merge feature. Так как в master после ответвления не было новых коммитов, Git выполнил слияние перемоткой (Fast-forward): указатель master просто передвинут на последний коммит feature, коммит слияния не создаётся. В git log --graph история линейная, master и feature указывают на один коммит (рисунок 6).

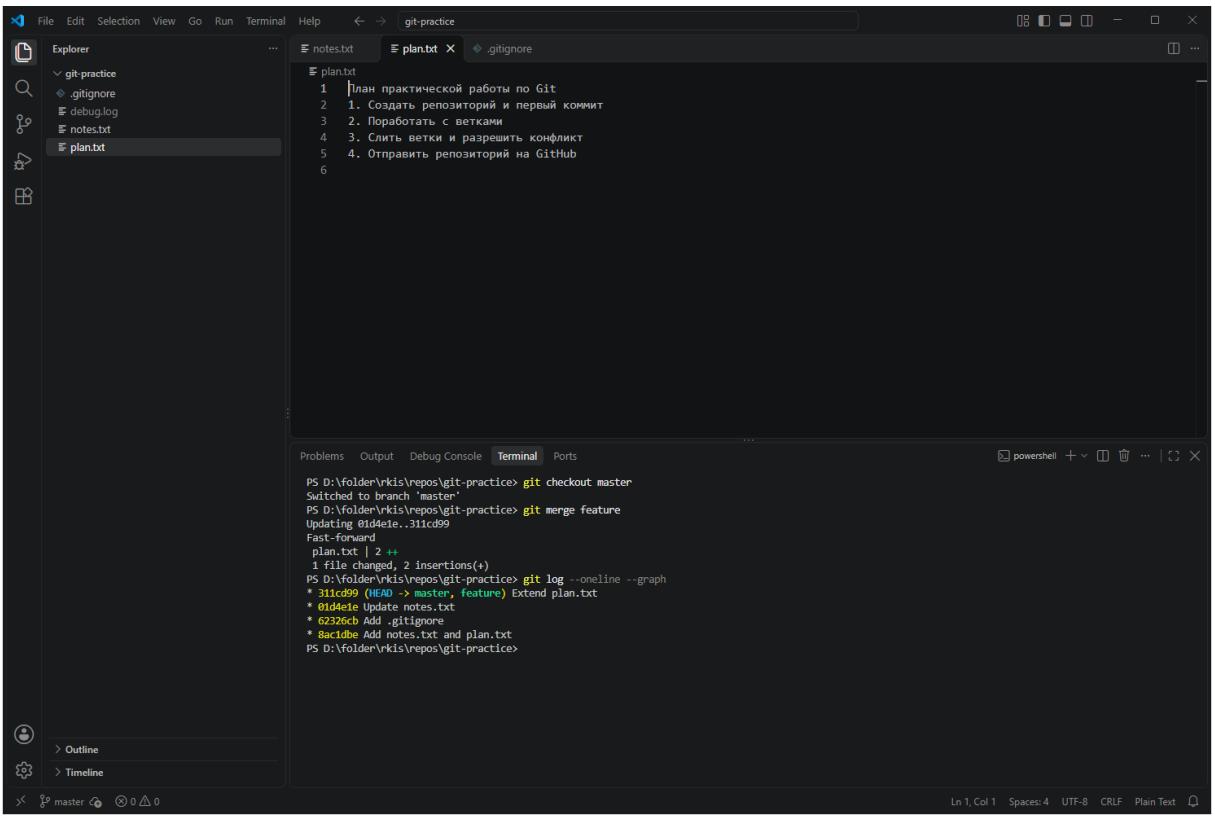


Рисунок 6 – Fast-forward слияние ветки feature в master

Шаг 7. Изменение в основной ветке

В ветке master в третьей строке notes.txt в список команд добавлена команда diff, изменение зафиксировано коммитом (рисунок 7).

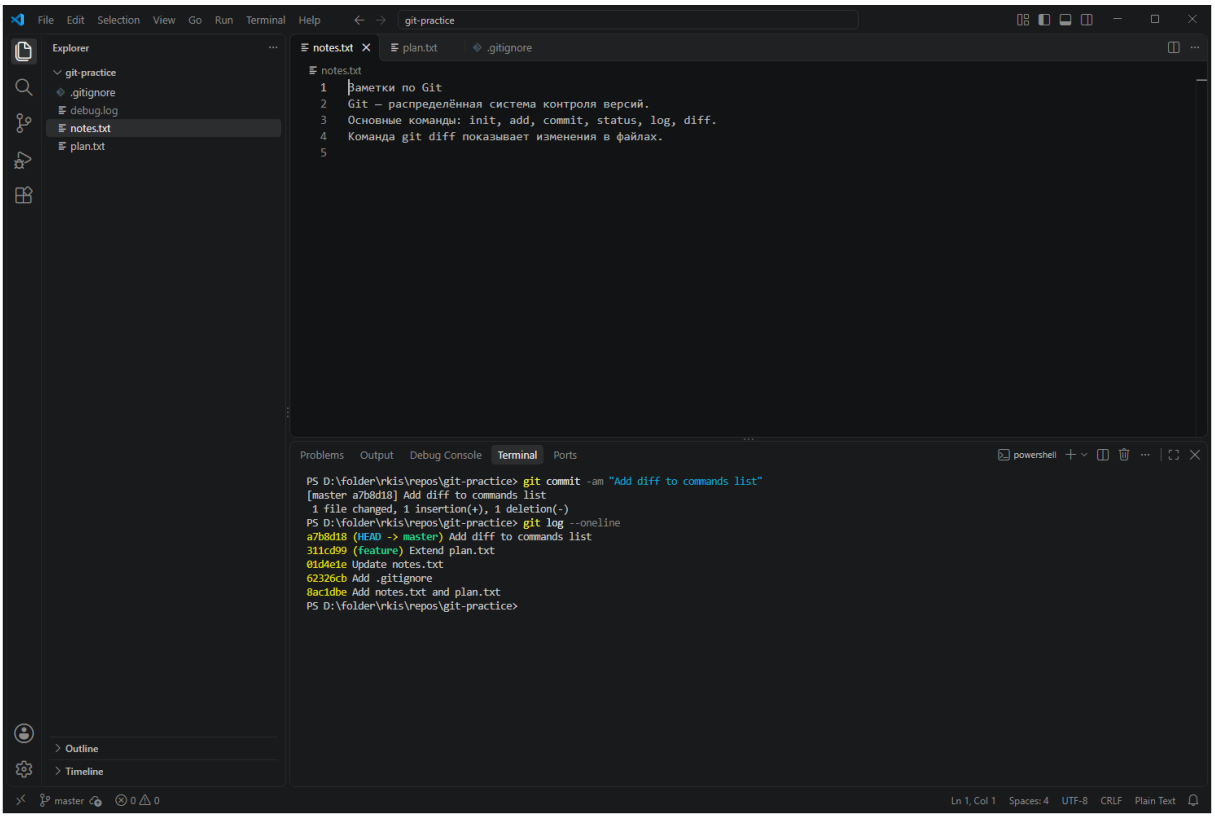


Рисунок 7 – Коммит в ветке master

Шаг 8. Конфликт слияния и его разрешение

В ветке feature та же третья строка notes.txt изменена по-другому (добавлены команды branch, checkout, merge), сделан коммит. При слиянии git merge feature из master возник конфликт: обе ветки изменили одну строку. Git остановил слияние и записал в файл обе версии между маркерами <<<<<<< HEAD, ======= и >>>>>>> feature; VS Code подсвечивает текущее (Current) и входящее (Incoming) изменение (рисунок 8).

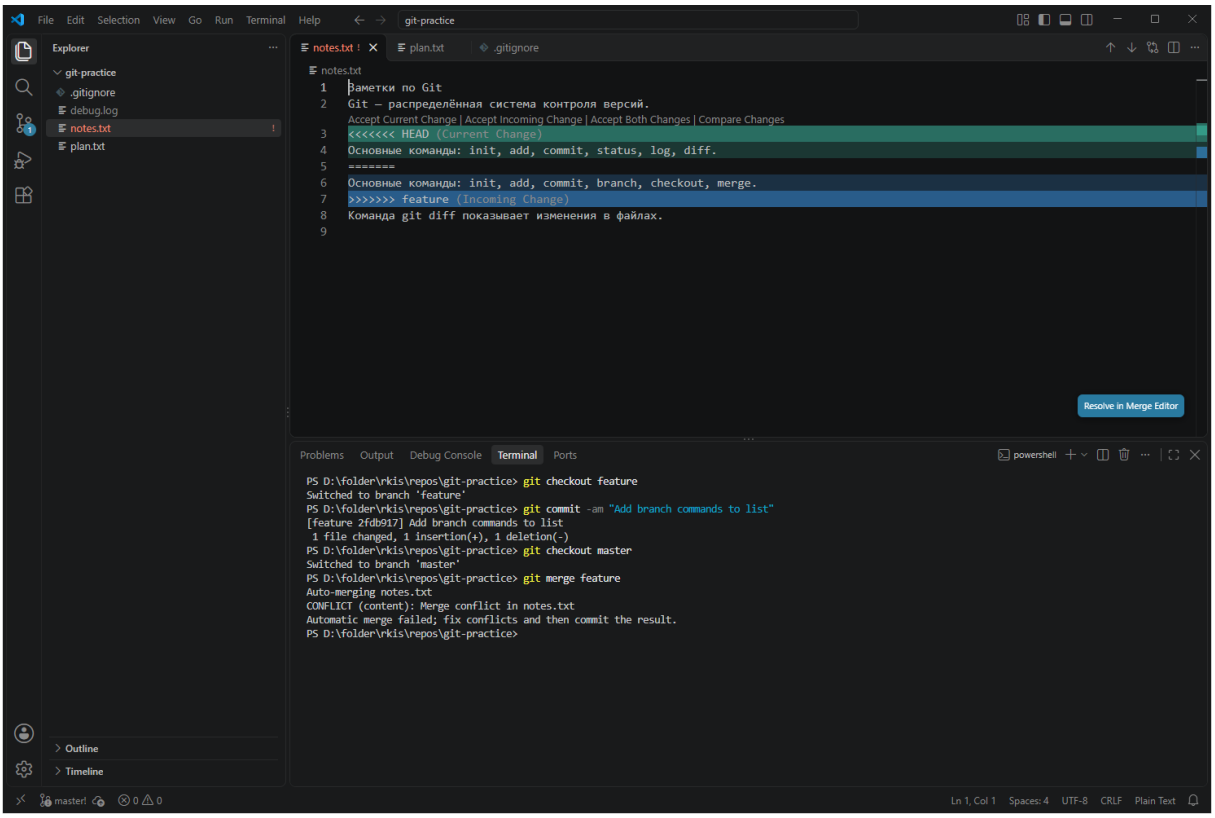


Рисунок 8 – Конфликт при слиянии веток

Конфликт разрешён вручную: маркеры удалены, в строке оставлен объединённый список команд из обеих веток. git status показывал файл как «both modified»; после git add notes.txt и git commit создан коммит слияния (рисунок 9).

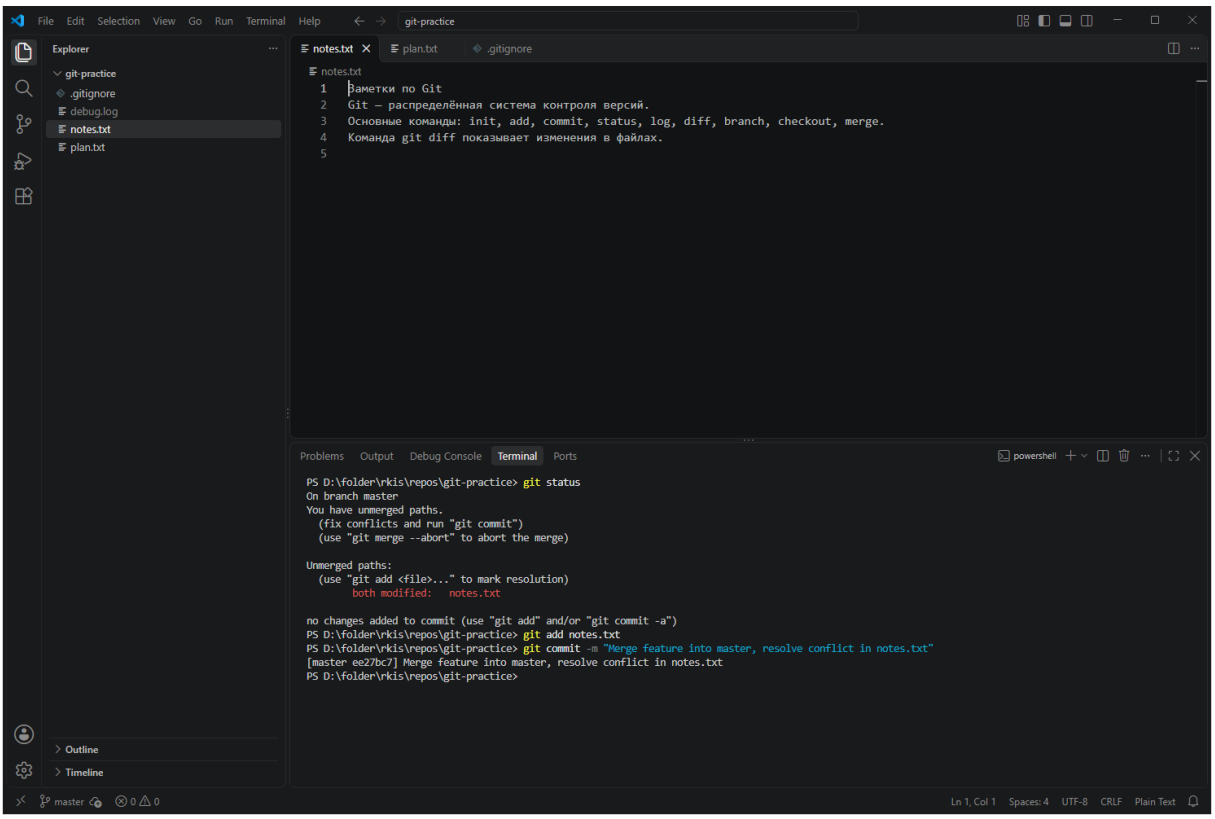


Рисунок 9 – Разрешение конфликта и коммит слияния

Шаг 9. История коммитов с ветвлениями

Команда git log --oneline --graph --all выводит историю в виде графа: после коммита «Extend plan.txt» ветки расходятся (по коммиту в master и в feature) и снова сходятся в коммите слияния (рисунок 10).

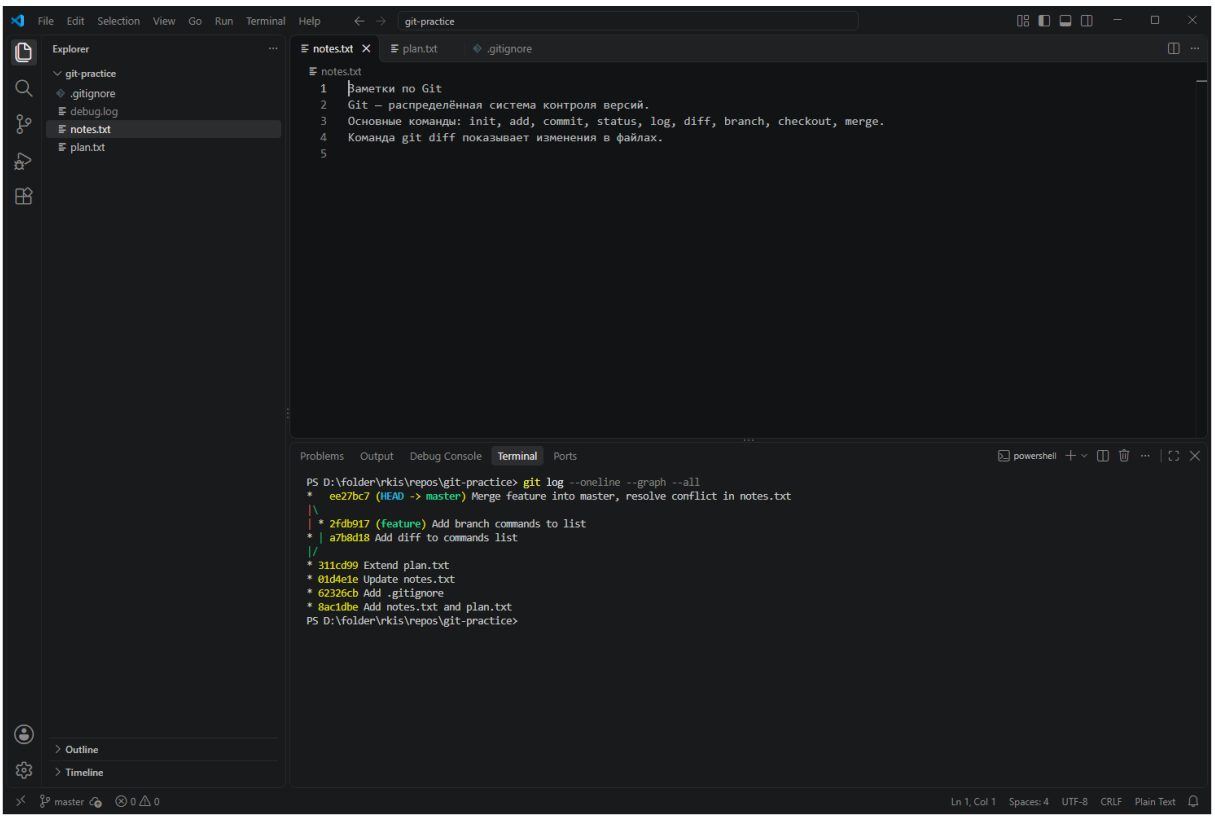


Рисунок 10 – История коммитов в виде графа

Шаг 10. Удалённый репозиторий на GitHub

На GitHub создан публичный репозиторий git-practice без начальных файлов (рисунки 11 и 12).

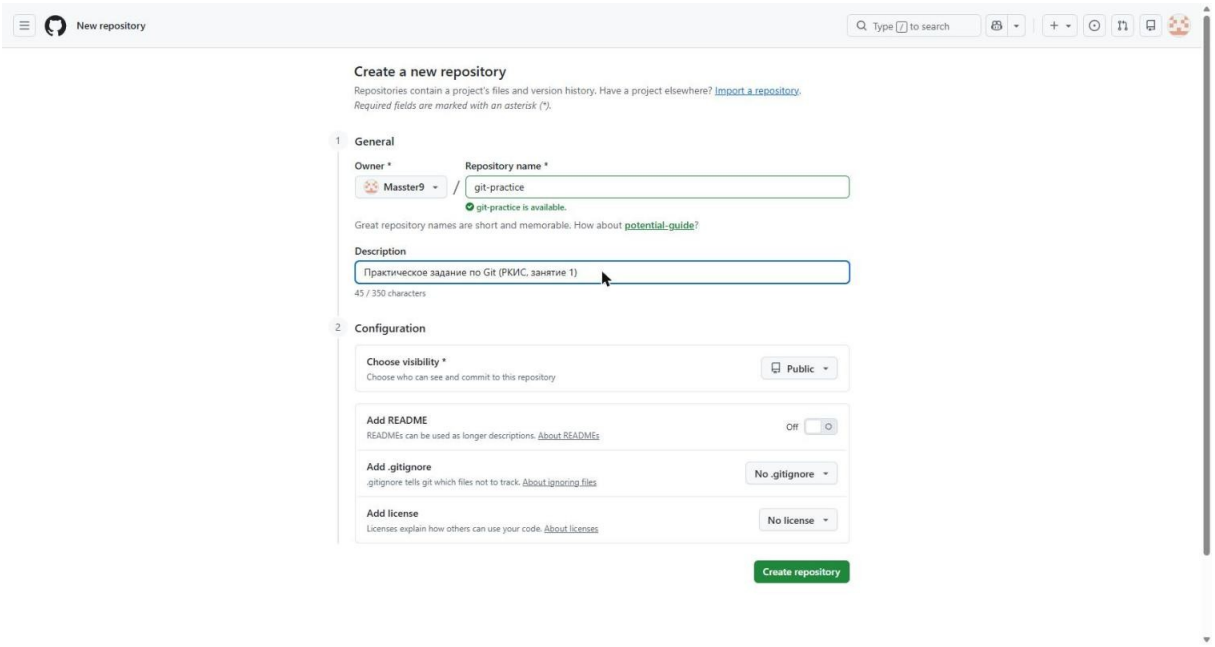


Рисунок 11 – Создание репозитория на GitHub

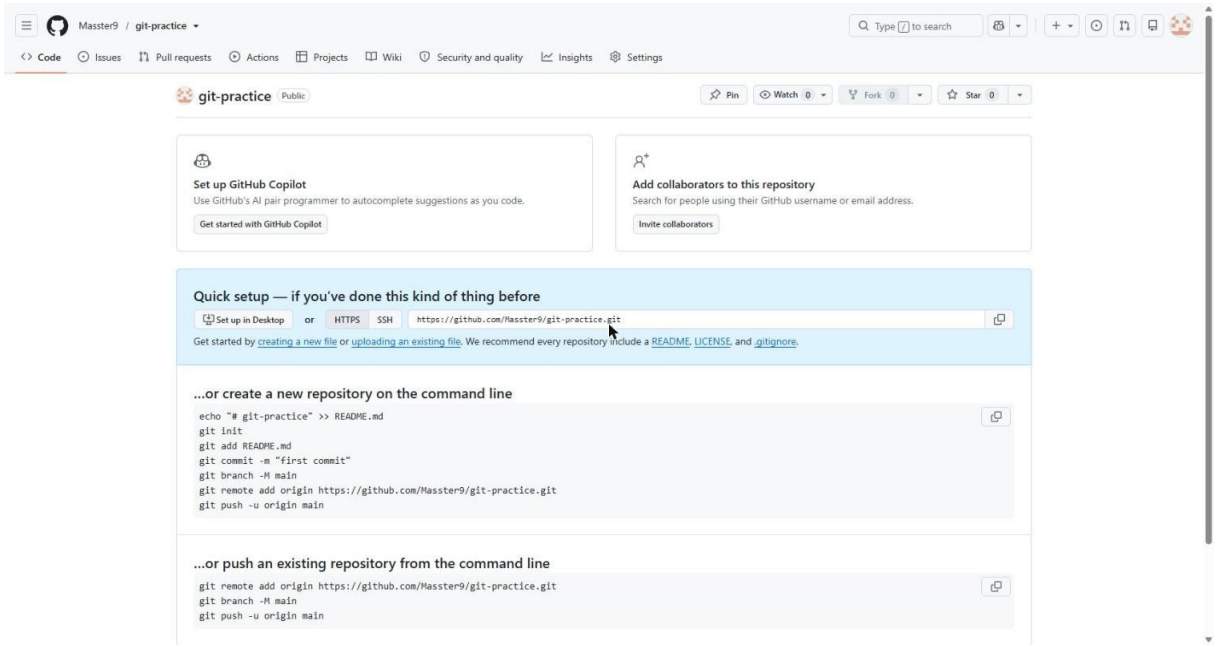


Рисунок 12 – Пустой удалённый репозиторий

Командой git remote add origin локальный репозиторий связан с удалённым, git remote -v показывает адреса для fetch и push. Командой git push -u origin master feature обе ветки отправлены на GitHub, ключ -u настроил их отслеживание (рисунок 13). Строка «fatal: Cannot prompt…» — сообщение менеджера учётных данных Git о том, что окно входа из терминала недоступно; вход выполнен через учётную запись GitHub в VS Code, и отправка прошла успешно. На GitHub появились файлы, две ветки и 7 коммитов (рисунок 14).

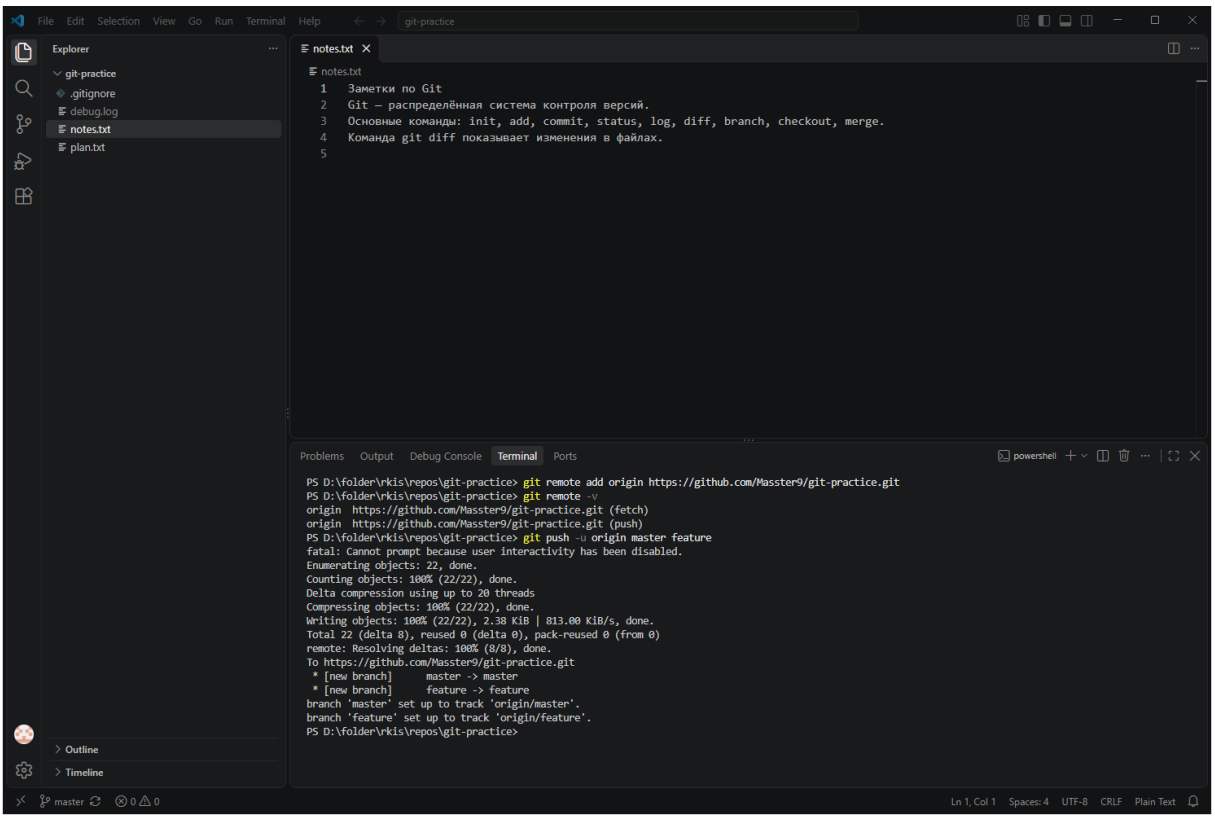


Рисунок 13 – Связь с удалённым репозиторием и git push

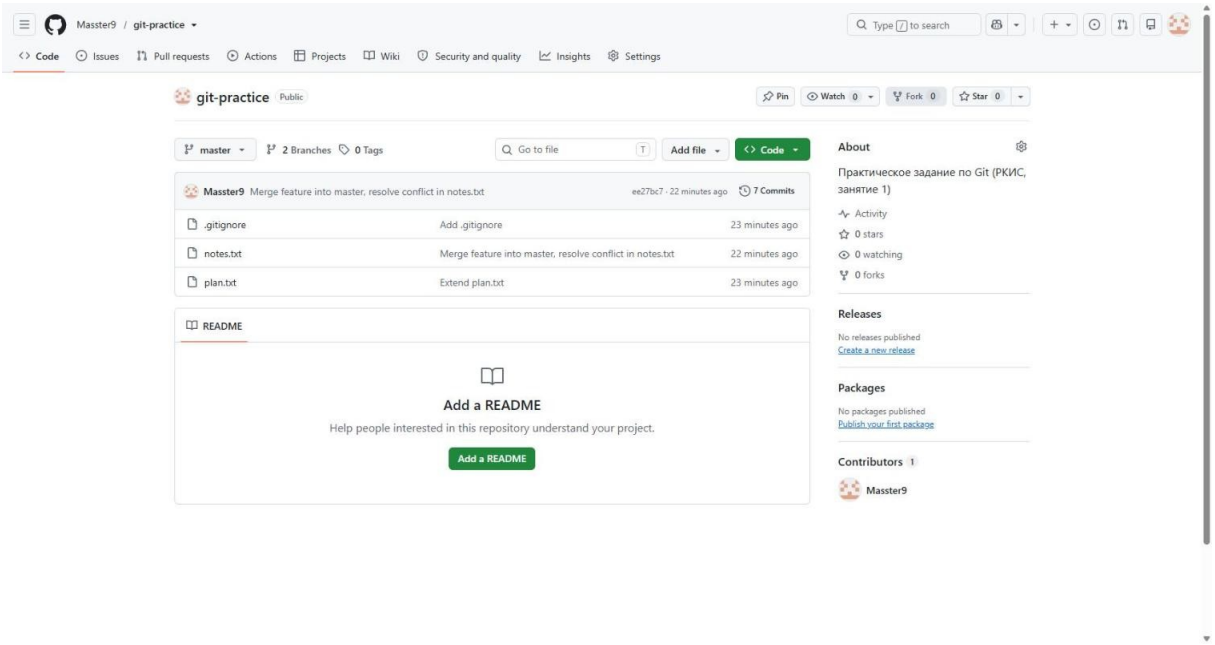


Рисунок 14 – Репозиторий на GitHub после отправки коммитов

Шаг 11. Создание README.md на GitHub

В веб-интерфейсе GitHub кнопкой «Add a README» создан файл README.md с описанием проекта и зафиксирован коммитом «Add README.md» прямо в ветку master (рисунки 15 и 16).

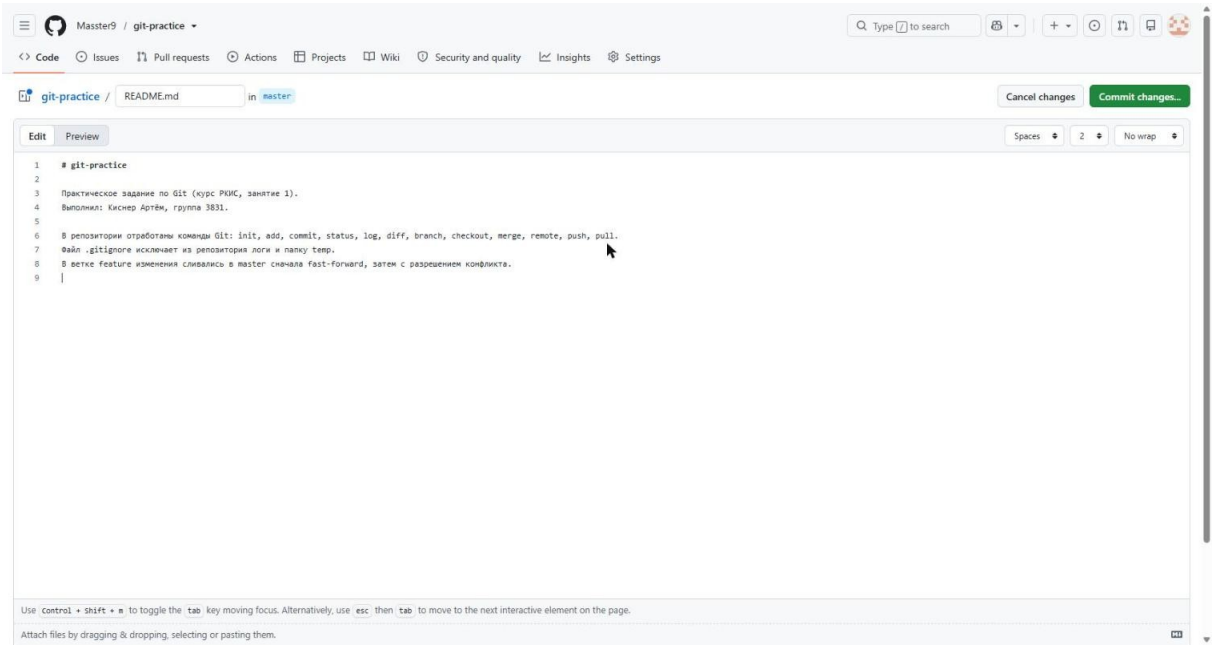


Рисунок 15 – Редактирование README.md на GitHub

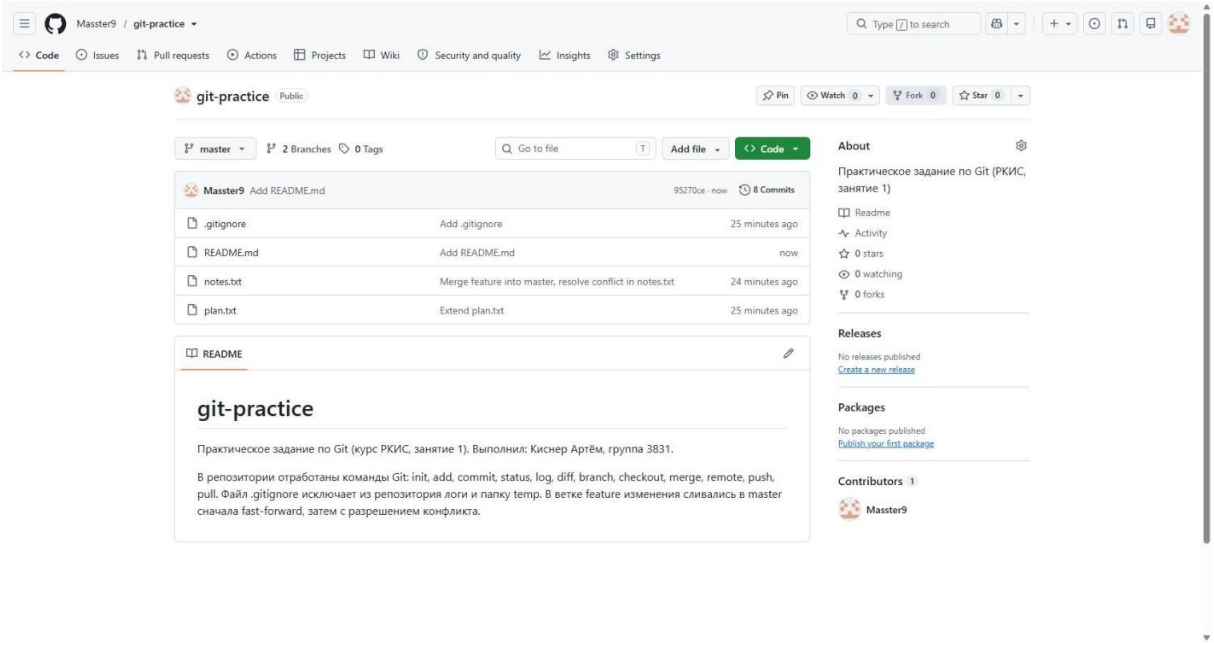


Рисунок 16 – README.md в репозитории на GitHub

Шаг 12. Получение изменений командой pull

Команда git pull скачала с GitHub новый коммит и слила его в локальную ветку master перемоткой — в папке проекта появился файл README.md. В истории ветки master и origin/master указывают на один коммит (рисунок 17).

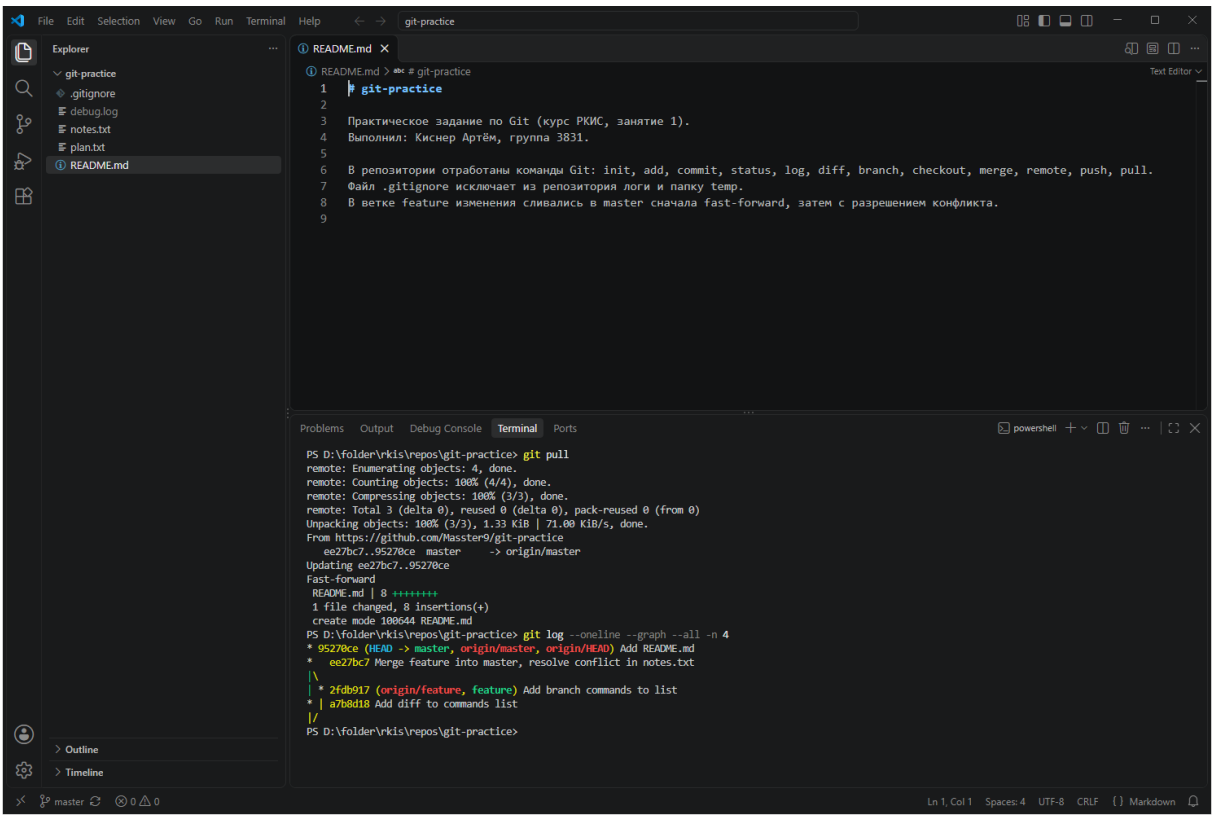


Рисунок 17 – Получение README.md командой git pull

Вывод

В ходе работы освоены основные операции Git: создание репозитория (init), индексирование и фиксация изменений (add, commit), просмотр состояния и истории (status, diff, log), игнорирование файлов через .gitignore. Отработана работа с ветками: создание и переключение (checkout -b, checkout), слияние перемоткой (fast-forward) и трёхстороннее слияние с ручным разрешением конфликта. Локальный репозиторий связан с удалённым на GitHub, изменения отправлены (push) и получены обратно (pull).